

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №9

Тема: Лінійний (послідовний) пошук

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Детально опанувати принцип роботи лінійного (послідовного) пошуку, навчитися оцінювати кількість порівнянь у найкращому, середньому та найгіршому випадках, ознайомитися з технікою оптимізації за допомогою елемента-сторожа (sentinel), набути навичок програмної реалізації та емпіричного дослідження алгоритму.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Принцип роботи. Лінійний пошук — найпростіший алгоритм пошуку елемента в масиві: елементи переглядаються послідовно, один за одним, від першого до останнього, доки не буде знайдено елемент з потрібним значенням або не скінчиться масив. Алгоритм не вимагає впорядкованості даних і застосовний до будь-якого набору.

2. Аналіз складності:

- **Найкращий випадок:** шуканий елемент — перший у масиві. Кількість порівнянь = 1. $O(1)$.
- **Найгірший випадок:** шуканий елемент — останній або відсутній у масиві. Кількість порівнянь = n . $O(n)$.
- **Середній випадок:** у середньому потрібно переглянути $n/2$ елементів (якщо елемент присутній і його позиція рівномірна). $O(n)$.

3. Оптимізація елементом-сторожем (sentinel). У класичній реалізації на кожній ітерації циклу виконується дві перевірки: (а) чи не вийшли за межі масиву ($i < n$) і (б) чи знайдено елемент ($a[i] == target$). Прийом сторожа дозволяє прибрати перевірку (а): шуканий елемент тимчасово записується в останню комірку масиву (яка потім звільняється), і цикл гарантовано зупиниться, знайшовши або справжній елемент, або сторожа в кінці — тобто на кожній ітерації лишається лише одна перевірка замість двох. Це не змінює асимптотичну складність $O(n)$, але зменшує кількість операцій у циклі приблизно вдвічі, що має практичне значення для дуже великих масивів.

4. Переваги й недоліки: головна перевага — універсальність (працює на будь-яких, зокрема неупорядкованих даних) і простота реалізації; головний недолік — низька ефективність порівняно з методами пошуку у впорядкованих структурах (бінарний пошук, хеш-таблиці), тому лінійний пошук доцільний лише для невеликих масивів або одноразових пошуків, коли витрати на попереднє сортування чи побудову структури не окупляться.

Хід роботи

Завдання 1. Ручна трасировка лінійного пошуку

Дано масив: [23, 8, 47, 15, 4, 62, 19, 31, 9, 55]. Виконати вручну лінійний пошук для трьох значень: 47 (елемент присутній ближче до початку), 9 (елемент присутній ближче до кінця), 100 (елемент відсутній). Для кожного випадку побудувати таблицю з номером порівняння та поточним елементом масиву.

Порядок виконання:

1. Для кожного цільового значення складіть таблицю зі стовпцями: "№ порівняння", "Індекс i", "Елемент a[i]", "a[i] == target?".
2. Починаючи з i=0, послідовно порівнюйте кожен елемент масиву з цільовим значенням, заповнюючи по одному рядку таблиці на кожне порівняння.
3. Зупиніться, щойно знайдете збіг (для 47 і 9), і зафіксуйте індекс знайденого елемента та загальну кількість порівнянь.
4. Для значення 100 продовжуйте таблицю до кінця масиву (усі 10 порівнянь), після чого зробіть висновок про відсутність елемента.
5. Порівняйте кількість порівнянь у трьох випадках з теоретичними оцінками найкращого, середнього і найгіршого випадків.

Завдання 2. Ручна трасировка алгоритму зі сторожем

Для того самого масиву виконати пошук значення 62 за допомогою техніки сторожа (sentinel), показавши, як тимчасово модифікується масив.

Порядок виконання:

1. Запишіть вихідний масив і окремо збережіть значення останнього елемента (55), яке буде тимчасово замінено.
2. Замініть останній елемент масиву на цільове значення 62 (сторож) і запишіть отриманий тимчасовий масив.
3. Виконайте пошук без перевірки меж масиву — просто порівнюйте a[i] з цільовим значенням, збільшуючи i, доки не знайдете збіг (він гарантовано знайдеться — або справжній елемент, або сторож на останній позиції).
4. Порахуйте кількість порівнянь, витрачених до знаходження збігу.
5. Визначте: якщо індекс знайденого збігу менший за довжину масиву мінус один — це справжнє входження; якщо збіг знайдено рівно на останній позиції — перевірте, чи це справжній елемент чи сторож (порівнявши зі збереженим оригінальним значенням).
6. Поверніть на місце оригінальне значення останнього елемента (55) у масиві.
7. Порівняйте кількість порівнянь зі сторожем із кількістю порівнянь у завданні 1 для аналогічного пошуку (без сторожа) і зробіть висновок про економію операцій.

Завдання 3. Програмна реалізація лінійного пошуку з сторожем

Реалізувати мовою Python звичайний лінійний пошук та його варіант з елементом-сторожем, перевірити коректність обох реалізацій.

Порядок виконання:

1. Напишіть функцію `linear_search(arr, target)`, яка в циклі `for i in range(len(arr))` порівнює кожен елемент з `target`, рахуючи кількість порівнянь; поверніть індекс (або `-1`) і кількість порівнянь.
2. Напишіть функцію `linear_search_sentinel(arr, target)`: створіть копію масиву, збережіть останній елемент у змінну `last`, замініть останній елемент на `target`.
3. Організуйте цикл `while True`, що порівнює `a[i]` з `target`, збільшуючи `i`, доки не знайде збіг (без перевірки виходу за межі).
4. Поверніть останній елемент масиву на місце (`a[n-1] = last`).
5. Перевірте: якщо індекс збігу менший за `n-1`, або (індекс дорівнює `n-1` і `last == target`) – елемент дійсно присутній; інакше поверніть `-1`.
6. Перевірте обидві функції на тестовому масиві для випадків "елемент присутній" та "елемент відсутній" — результати мають збігатися.

Приклад реалізації мовою Python:

```
python
def linear_search(arr, target):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        if arr[i] == target:
            return i, comparisons
    return -1, comparisons

def linear_search_sentinel(arr, target):
    a = arr.copy()
    n = len(a)
    last = a[n-1]
    a[n-1] = target # сторож
    i = 0
    comparisons = 0
    while True:
        comparisons += 1
        if a[i] == target:
            break
        i += 1
    a[n-1] = last
    if i < n-1 or a[n-1] == target:
        return i, comparisons
    return -1, comparisons

arr = [5, 3, 8, 1, 9, 2, 7]
print(linear_search(arr, 9))      # (4, 5)
print(linear_search_sentinel(arr, 9)) # (4, 5)
print(linear_search(arr, 100))    # (-1, 7)
print(linear_search_sentinel(arr, 100))# (-1, 7)
```

Результат виконання програми:

```
(4, 5)
(4, 5)
(-1, 7)
(-1, 7)
```

Обидві реалізації повертають однакові результати — коректність підтверджено.

Завдання 4. Емпіричне дослідження кількості порівнянь

Дослідити середню (для випадково обраного присутнього елемента) та найгіршу (елемент відсутній) кількість порівнянь лінійного пошуку для масивів різного розміру, порівняти з теоретичними оцінками $n/2$ та n .

Порядок виконання:

1. Для кожного розміру n з набору $\{1000, 5000, 10000, 50000\}$ згенеруйте масив з n випадкових (перемішаних) чисел.
2. Виконайте 200 випробувань: на кожному оберіть випадковий елемент масиву як ціль пошуку, виконайте `linear_search` і накопичте кількість порівнянь.
3. Обчисліть середню кількість порівнянь, поділивши суму на кількість випробувань (200).
4. Виконайте один пошук елемента, якого гарантовано немає в масиві (наприклад, -1), щоб отримати кількість порівнянь для найгіршого випадку.
5. Виведіть для кожного n : середню кількість порівнянь (має бути близько до $n/2$) та найгіршу кількість (має дорівнювати рівно n).
6. Порівняйте отримані експериментальні значення з теоретичними формулами та зробіть висновок про їх узгодженість.

Приклад реалізації мовою Python:

```
python
import random

sizes = [1000, 5000, 10000, 50000]
for n in sizes:
    arr = list(range(n))
    random.shuffle(arr)
    total = 0
    trials = 200
    for _ in range(trials):
        t = random.choice(arr)
        _, c = linear_search(arr, t)
        total += c
    avg = total / trials
    _, c_worst = linear_search(arr, -1) # відсутній елемент
    print(f"n={n}: середня к-ть порівнянь = {avg:.1f}, найгірший випадок = {c_worst}")
```

Результат виконання програми:

$n=1000$: середня к-ть порівнянь(знайдено)=482.2, найгірший випадок(не знайдено)=1000
 $n=5000$: середня к-ть порівнянь(знайдено)=2531.1, найгірший випадок(не знайдено)=5000
 $n=10000$: середня к-ть порівнянь(знайдено)=5116.1, найгірший випадок(не знайдено)=10000
 $n=50000$: середня к-ть порівнянь(знайдено)=25252.5, найгірший випадок(не знайдено)=50000

Висновок за результатами: середня кількість порівнянь дуже близька до $n/2$ (наприклад, для $n=50000$ отримано 25252,5 — практично точно $n/2=25000$), а найгірший випадок точно дорівнює n . Це повністю підтверджує теоретичну оцінку складності лінійного пошуку $O(n)$ у середньому й найгіршому випадках.

Контрольні питання

1. Опишіть принцип роботи лінійного (послідовного) пошуку.

2. Які вимоги до вхідних даних висуває лінійний пошук (щодо впорядкованості)?
3. Обчисліть кількість порівнянь у найкращому, середньому та найгіршому випадках лінійного пошуку.
4. У чому суть техніки елемента-сторожа? Яку саме перевірку в циклі вона усуває?
5. Чи змінює сторож асимптотичну складність алгоритму? Що саме він оптимізує?
6. У яких ситуаціях застосування лінійного пошуку виправдане, попри його низьку ефективність порівняно з бінарним пошуком?

Зміст звіту

1. Тема, мета роботи.
2. Розгорнуті ручні трасировки (таблиці) до завдань 1 і 2.
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Висновки.